Software/ML Taskphase Report

Conceptual Report on Tasks 1–5

Submitted by

Tanya Misra

Roll Number: 251090052106

Branch: Computer Science and Engineering (CSE)

Domain: Software/ML

Date of Submission: 28th June 2026

Institution:
Manipal Institute of Technology (MIT), Manipal Academy of Higher Education (MAHE)
Manipal, Karnataka, India

GitHub Repository:
https://github.com/TanTheta017/Synergy_TP

Abstract:

This report presents the theoretical concepts and practical knowledge gained during the completion of Tasks 1 to 5 of the Software/ML Domain Taskphase. The work covered version control using Git and GitHub, Python programming fundamentals, manual CSV parsing, data analysis using Pandas, data cleaning techniques, validation of processed datasets, and data visualization using Matplotlib. The report explains the purpose of each concept, the reasoning behind important implementation and data-processing decisions, and how these techniques contribute to reliable software development and data analysis. It also highlights the differences between manual and library-based approaches for handling structured data and emphasizes the importance of data quality before visualization. Rather than focusing on source code, the report provides a conceptual understanding of the tools, methods, and workflows used throughout the task phase. Overall, the completed tasks strengthened practical skills in Python programming, data processing, and maintaining organized software projects using modern development practices.

Introduction:

The Software/ML Domain Taskphase was designed to provide practical experience with essential concepts used in software development and data processing. Through a series of structured tasks, participants

developed skills in version control, Python programming, file handling, data processing, data cleaning, validation, and data visualization. Each task built upon the previous one, creating a complete workflow that demonstrated how raw data can be transformed into meaningful insights.

The task phase began with learning Git and GitHub for project organization and version control, followed by the application of Python fundamentals for implementing modular and reusable programs. Participants then explored manual CSV parsing to understand how structured data is created from plain text before using the Pandas library to simplify data analysis. The next stage focused on cleaning and validating datasets by handling missing values, duplicate records, inconsistent categories, incorrect data types, and invalid entries. Finally, the cleaned data was visualized using Matplotlib to present trends and comparisons in a clear graphical format.

This report provides the theoretical background behind the concepts covered in Tasks 1 to 5. Rather than describing the implementation code, it explains the purpose of each tool, the reasoning behind important data-processing decisions, and the significance of following systematic software development practices. The report also highlights how these concepts work together to create reliable, maintainable, and efficient data-processing workflows commonly used in software engineering and machine learning applications.

Development Environment and Version Control:

A suitable development environment is essential for building, testing, and maintaining software projects efficiently. During this task phase, Python was used as the primary programming language, while Visual Studio Code served as the development environment for writing and executing programs. Git was used as the version control system to track changes made throughout the project, and GitHub was used to host the repository online and maintain a backup of all completed work.

Version control played an important role in managing the project. Git records every modification made to the source code, allowing developers to restore previous versions whenever required. Regular commits created checkpoints throughout development, making it easier to monitor progress and debug errors. GitHub complemented Git by providing an online repository where the project could be stored, shared, and submitted for evaluation.

Maintaining a clean repository structure improved the organization of the project. Separate folders were created for each task, while common files such as README.md, requirements.txt, and .gitignore were kept in the root directory. This organization made the repository easier to navigate and ensured that all required files could be located quickly.

To avoid dependency conflicts, the project was developed inside a Python virtual environment. A virtual environment creates an isolated workspace containing only the

libraries required for a particular project. This ensures that package versions remain consistent and prevents interference from software installed for other projects.

The requirements.txt file was used to record all external Python packages required to run the project. This allows another user to recreate the same environment by installing the listed dependencies, ensuring that the project behaves consistently across different systems.

Another important file included in the repository was .gitignore, which specifies files and folders that should not be tracked by Git. Temporary files, cache folders, compiled Python files, and virtual environments were excluded because they can be regenerated automatically and are not necessary for sharing the project.
Using .gitignore keeps the repository clean and ensures that only essential project files are uploaded to GitHub.

Python and File Handling Concepts:

Python provided the foundation for implementing all the tasks completed during the Software/ML Domain Taskphase. Its simple syntax, extensive standard library, and support for modular programming made it suitable for developing programs for CSV parsing, data cleaning, validation, and visualization. Throughout the project, several fundamental Python concepts were applied to create organized, reusable, and maintainable code.

Functions were used to divide the programs into smaller, independent units, with each function performing a specific task such as reading input files, cleaning data, generating summaries, validating records, or creating visualizations. This modular approach improved readability, reduced code repetition, and made debugging and future modifications easier.

Python's built-in data structures, particularly lists and dictionaries, played an important role in processing the datasets. Lists were used to store collections of records and iterate through multiple values efficiently, while dictionaries represented individual student records using key-value pairs. Using dictionaries made the data more meaningful because values could be accessed through descriptive field names instead of numerical positions.

File handling was another essential concept used throughout the task phase. The project involved reading input data from CSV files, writing processed summaries to JSON files, generating Markdown reports, and saving graphical outputs as PNG images. The use of the with open() context manager ensured that files were opened and closed safely, reducing the possibility of resource leaks and improving program reliability.

Exception handling improved the robustness of the programs by allowing unexpected situations, such as missing files, incorrect file paths, or invalid data, to be handled gracefully instead of causing the program to terminate unexpectedly. Providing meaningful error messages helps users identify problems while maintaining the stability of the application.

Type hints were also used to improve code readability by specifying the expected data types of function parameters and return values. Although Python is dynamically typed, type hints make programs easier to understand, assist code editors in detecting potential errors, and encourage better programming practices, especially in larger software projects.

CSV Parsing and Pandas:

CSV (Comma-Separated Values) is one of the most commonly used formats for storing tabular data. It organizes information into rows and columns, where each row represents a record and each column represents a specific attribute. Since CSV files store all values as plain text, data often requires type conversion before it can be processed or analyzed. During the task phase, CSV files were used as the primary source of input data for analysis and cleaning.

Manual CSV parsing involves reading the CSV file using Python's built-in file handling functions without relying on external libraries. The file is processed line by line, the header is separated from the data rows, and each row is converted into a dictionary using the column names as keys. Numeric values such as scores are converted to the appropriate data types, while invalid or malformed rows are handled carefully to prevent errors. This approach provides a better understanding of how structured data is created from raw text and helps build a strong foundation in file processing.

Pandas is a powerful Python library designed for data manipulation and analysis. It allows CSV files to be loaded directly into a DataFrame, where the data is organized into rows and columns with built-in support for filtering, sorting, grouping, and statistical operations. Compared to manual parsing, Pandas requires significantly less code and provides optimized functions for handling large datasets efficiently.

Both manual CSV parsing and Pandas were used during the task phase to generate the same summary statistics, allowing a comparison between the two approaches. Manual parsing offers greater control over the processing of each row and helps in understanding the internal structure of CSV files. However, it requires more programming effort and becomes less efficient as the size of the dataset increases. Pandas simplifies most of these operations through built-in methods, making the code shorter, easier to maintain, and more suitable for real-world software development and machine learning applications.

The comparison demonstrated that manual parsing is valuable for learning the underlying concepts of data processing, whereas Pandas is the preferred choice for practical applications due to its efficiency, reliability, and extensive functionality.

Data Cleaning:

Data cleaning is an essential step in data processing because raw datasets often contain errors, inconsistencies, and incomplete information that can affect the accuracy of analysis. Before performing

calculations or creating visualizations, the dataset must be cleaned to ensure that it is reliable, consistent, and suitable for further processing. During Task 4, various data cleaning techniques were applied to improve the quality of the student dataset.

A. Missing Values

Missing values occur when information is absent from one or more fields in a dataset. Such values can lead to incorrect calculations, inaccurate visualizations, or unreliable conclusions if they are ignored. Instead of removing all incomplete records, appropriate rules should be defined to handle missing data. Depending on the type of information, missing values may be replaced with suitable values, estimated using other records, or retained if they do not significantly affect the analysis. Documenting these decisions ensures that the cleaning process remains transparent and reproducible.

B. Duplicate Records

Duplicate records are multiple entries representing the same data. If left in the dataset, they can distort statistics such as averages, counts, and percentages. Therefore, duplicate rows should be identified and removed before analysis so that each record appears only once. Removing duplicates improves the accuracy of the dataset and prevents biased results.

C. Data Type Conversion

Data imported from CSV files is generally stored as text, even when it represents numerical information. Before performing calculations, values such as scores, attendance percentages, and study hours must be converted into appropriate numeric data types. In some cases, textual numbers such as "nine" or "two" also need to be converted into integers. Correct data types ensure that mathematical operations can be performed accurately.

D. Unit Conversion

Measurements are often recorded using different units or formats. For example, height may be stored in both metres and centimetres, while weight values may contain different spacing or unit representations. Converting all measurements into a single standard unit ensures consistency throughout the dataset and makes comparisons more meaningful.

E. Category Standardization

Categorical values frequently appear in different formats even though they represent the same category. Examples include values such as "ML", "ml", and "Machine Learning", which all refer to the same domain. Similarly, submission status may appear as "Yes", "Y", or "yes". Standardizing these values into a single consistent format improves grouping, filtering, and statistical analysis while reducing inconsistencies within the dataset.

F. Invalid Values

Datasets may contain values that fall outside the acceptable range, making them invalid for analysis. Examples include attendance percentages below 0% or above 100%, negative measurements, or incorrect numerical entries. Such values must be corrected, replaced, or removed according to

clearly defined rules. Handling invalid data prevents misleading results and improves the overall reliability of the dataset.

G. Validation

After the cleaning process is complete, validation is performed to verify that the dataset satisfies all required conditions. Validation checks ensure that student IDs are unique, numerical fields contain valid values, attendance percentages remain within the range of 0 to 100, domain names belong to the predefined categories, and important fields do not contain missing values. Validation serves as the final quality assurance step before data analysis and visualization, ensuring that the cleaned dataset is accurate, consistent, and ready for further use.

Visualization:

Data visualization is the process of representing data graphically so that patterns, trends, and relationships can be understood more easily. After cleaning and validating the dataset, visualization was performed using the Matplotlib library to convert numerical information into meaningful graphs. Visual representations make it easier to interpret results than large tables of numbers and help communicate findings effectively.

Three different plots were generated during this task phase. The first was a **bar chart** showing the average score for each domain. This chart provided a clear comparison of student performance across different domains and helped identify which domain had the highest and lowest average scores.

The second visualization was a **scatter plot** illustrating the relationship between attendance percentage and student scores. Each point on the graph represented one student's record, making it possible to observe whether higher attendance was associated with better academic performance. Scatter plots are useful for identifying trends, patterns, and unusual observations within a dataset.

The third visualization was another **bar chart** showing the number of students who submitted and did not submit their tasks. This graph provided a simple comparison of submission status and made it easy to understand the overall participation of students.

While creating the visualizations, appropriate titles, axis labels, and spacing were included to improve readability. The figures were saved as PNG files so that they could be reused in reports and presentations without rerunning the program. Proper labeling and formatting ensure that graphs are easy to understand and accurately communicate the information being presented.

Overall, visualization represented the final stage of the data processing workflow. After collecting data, cleaning inconsistencies, validating records, and generating summaries, graphical representation transformed the processed data into clear

and meaningful insights. This demonstrated the importance of visualization as an effective tool for analysing and communicating information in software development, data science, and machine learning applications.

Reflection:

Throughout the task phase, the dataset underwent several stages of transformation before meaningful insights could be obtained. Initially, the raw CSV file contained inconsistencies such as duplicate records, missing values, mixed measurement units, inconsistent category names, and invalid data entries. In this form, the dataset was unsuitable for reliable analysis because these issues could lead to inaccurate calculations and misleading results.

After applying data cleaning techniques, the dataset became standardized and consistent. Duplicate records were removed, missing and invalid values were handled using defined rules, categories were normalized, units were converted into common formats, and data types were corrected. Validation was then performed to ensure that the cleaned dataset met all required quality standards and was ready for analysis.

Finally, the cleaned data was used to generate visualizations that presented the information in a clear and meaningful way. Graphs made it easier to identify patterns, compare domain performance, and observe relationships between variables such as attendance and scores. This progression from raw data to cleaned data and finally to visual representation demonstrated the importance of systematic data processing. The task phase highlighted that accurate analysis depends not only on programming skills but also on maintaining data quality and following organized software development practices.

Conclusion:

The Software/ML Domain Taskphase provided a strong foundation in the fundamental concepts of software development and data processing. Through the completion of Tasks 1 to 5, practical knowledge was gained in version control using Git and GitHub, Python programming, file handling, manual CSV parsing, data analysis with Pandas, data cleaning, dataset validation, and data visualization using Matplotlib.

Each task demonstrated the importance of following a structured workflow while solving real-world data processing problems. The experience highlighted that accurate analysis depends not only on writing functional code but also on maintaining clean, consistent, and validated data. It also emphasized the value of organizing projects using proper repository structures, reusable programming practices, and clear documentation.

Overall, the task phase enhanced both technical knowledge and problem-solving skills by combining theoretical concepts with practical implementation. The skills developed during these tasks provide a solid foundation for future work in software engineering, data science, and machine learning, where reliable data processing and

well-organized software development practices are essential.

References:

1. Python Software Foundation. *Python 3 Documentation*. https://docs.python.org/3/
2. Pandas Development Team. *Pandas Documentation*. https://pandas.pydata.org/docs/
3. Matplotlib Development Team. *Matplotlib Documentation*. https://matplotlib.org/stable/
4. Git Project. *Git Documentation*. https://git-scm.com/doc
5. GitHub Docs. *GitHub Documentation*. https://docs.github.com/
6. Visual Studio Code Documentation. *Visual Studio Code*. https://code.visualstudio.com/docs

Table :

Task	***Main Concept Learned***
Task 1	Git, GitHub, repository structure, commits, branches, and version control
Task 2	Python fundamentals, functions, data structures, file handling, exceptions, and type hints
Task 3	Manual CSV parsing, Pandas, JSON generation, and data summarization
Task 4	Data cleaning, missing value handling, duplicate removal, type conversion, validation, and normalization
Task 5	Data visualization using Matplotlib and interpretation of graphical results

Flowchart:

Raw CSV Data → Read CSV File → Manual Parsing / Pandas → Data Cleaning → Data Validation → Summary Generation → Data Visualization → Insights & Analysis